Autonomous Legal Researcher Agent

Course Name: Agentic AI

Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	VISHAL RATHOD	EN22CS306059
2.	PALAK GUPTA	EN22CS306036
3.	JAY PRAKASH SINGH	EN22CS304031
4.	RAJVEER MUKATI	EN22CS304051
5.	KOMAL JADHAV	EN22CS303028
6.	DIYA GOYAL	EN22CS304025

Group Name: G11D11

Project Number: AAI-37

Industry Mentor Name: Mr. Ashwin Krishnan Sir

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: 2025-26

1. Problem Statement & Objectives

1.1 Problem Statement

The legal domain contains a large amount of complex and continuously changing information, making it difficult for users to quickly find accurate legal guidance. Traditional search methods often provide scattered and unstructured results. This project aims to solve this problem by developing an Agentic Legal AI Assistant that can automatically research legal queries, retrieve relevant information from memory and web sources, and generate a concise, easy-to-understand legal response.

1.2 Project Objectives

- To build an AI-powered legal assistant for answering legal questions
- To use a **multi-agent workflow** for research and summarization
- To retrieve legal data from **web sources and vector memory**
- To provide concise and structured legal responses
- To store useful responses for future reuse
- To create a user-friendly chat-based interface
- To provide legal responses in **simple and understandable language**.
- To store useful query results for future reuse and faster responses.

1.3 Scope of the Project

The scope of this project includes:

- **Accepting legal questions from users through a chat interface.**
- **Searching stored memory and online legal sources.**
- **Summarizing legal information using AI agents.**
- **Displaying accurate and concise responses.**
- **Maintaining reusable memory for future queries.**

2. Proposed Solution

2.1 Key features

- Legal query answering system
- Multi-agent architecture
- Memory-based retrieval
- Web search integration
- AI-powered summarization
- Chat-based user interface
- Reusable legal knowledge storage
- Fast response generation
- Reusable memory system

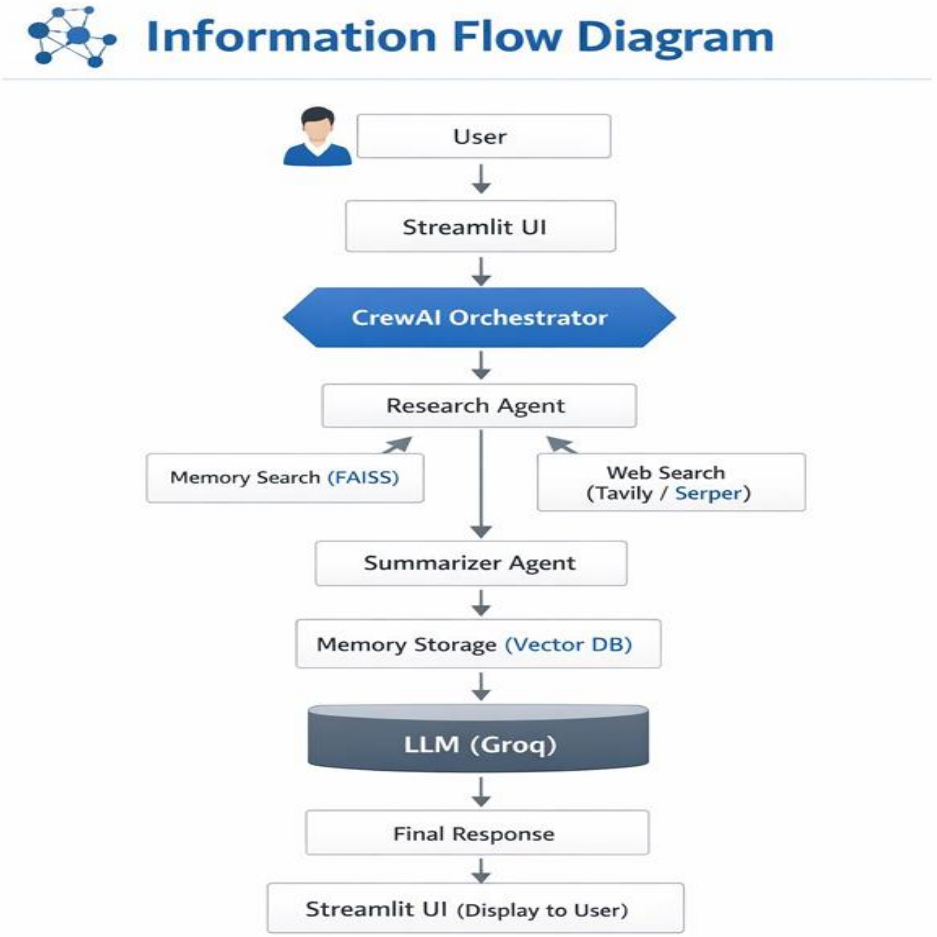
2.2 Overall Architecture / Workflow

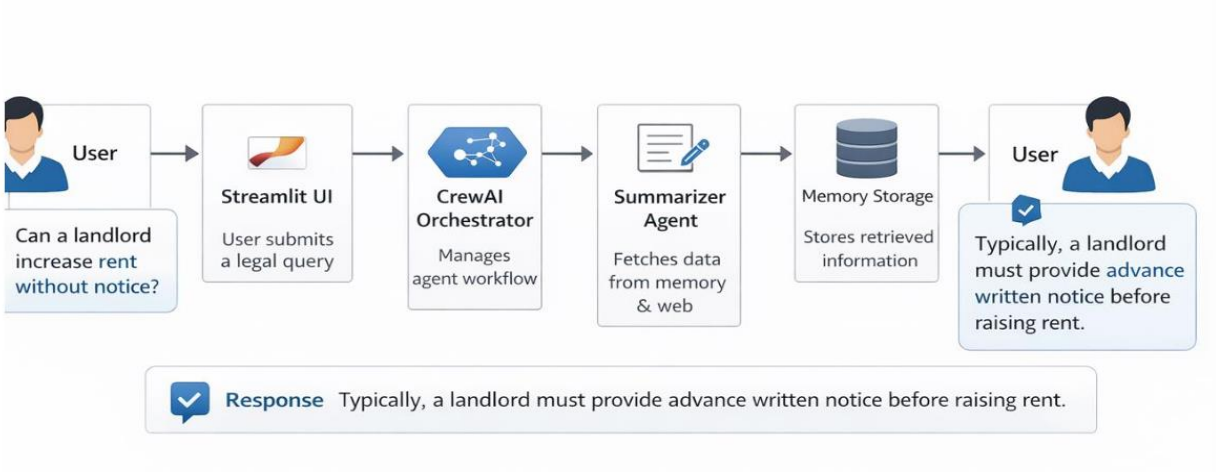
The project follows a **multi-agent workflow architecture**:

- User submits a legal question through the **Streamlit UI**.
- **CrewAI Orchestrator** manages the workflow.
- **Research Agent** searches legal information from:
  - **FAISS Memory**
  - **Web Search API**
- The collected data is sent to the **Summarizer Agent**.
- The summarizer generates a simple final response.
- Useful information is stored back into memory for future use.
- Final answer is displayed to the user.
- This workflow improves response quality by combining **memory retrieval, web search, and LLM reasoning**.

➢ Process Flow

- **User Input: -** User submits a legal query through the interface.
- **Task Orchestration: -** Query is processed by CrewAI to manage agent workflow.
- **Research Phase: -** Research Agent gathers data from memory and web sources
- **Summarization: -** Summarizer Agent converts data into a clear and structured response.
- **Output & Storage :-** Final answer is displayed and stored for future reuse


Information Flow Diagram
User
Streamlit UI
CrewAI Orchestrator
Research Agent
Memory Search (FAISS)
Web Search (Tavily / Serper)
Summarizer Agent
Memory Storage (Vector DB)
LLM (Groq)
Final Response
Streamlit UI (Display to User)


User
Can a landlord increase rent without notice?
Streamlit UI
User submits a legal query
CrewAI Orchestrator
Manages agent workflow
Summarizer Agent
Fetches data from memory & web
Memory Storage
Stores retrieved information
User
Typically, a landlord must provide advance written notice before raising rent.
Response Typically, a landlord must provide advance written notice before raising rent.

➢ Step-by-Step Information Flow

- User enters a legal query through the Streamlit interface
- Query is sent to the CrewAI orchestrator for task handling
- Research Agent checks vector memory (FAISS) for relevant past data
- If required, web search (Tavily / Serper) is performed to fetch updated information
- Retrieved data is analyzed and structured by the Summarizer Agent
- Important insights are stored in memory for future queries
- Final response is generated using Groq LLM 8. Response is displayed to the user via the Streamlit UI

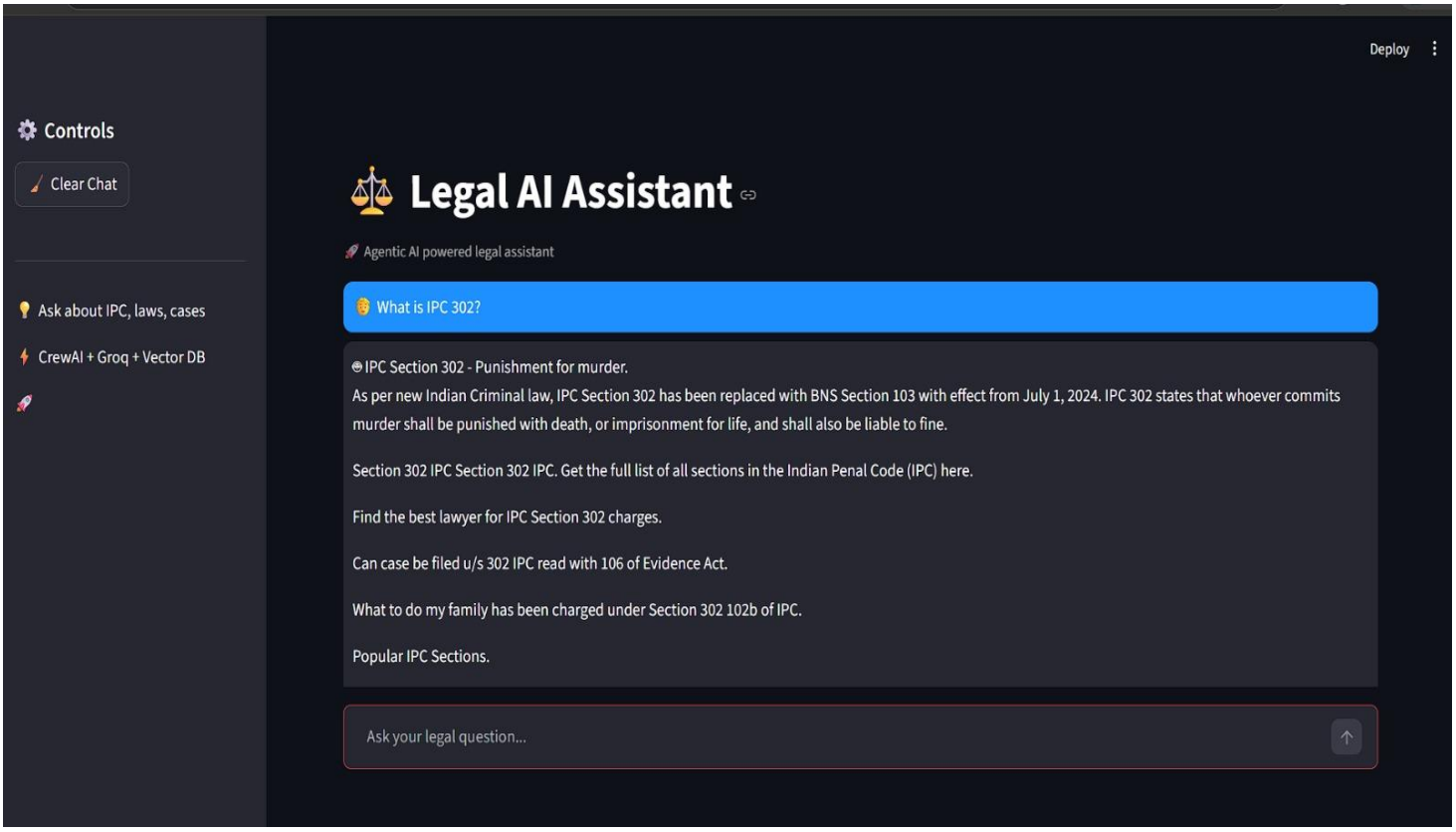
2.3 Tools & Technologies Used

Category	Tool / Technology
Programming	Python
Frontend	Streamlit
Multi-Agent Framework	CrewAI
Language Model (LLM)	Groq API (Llama 3.1 8B)
Vector Database	FAISS
Embeddings	SentenceTransformers
Web Search	Tavily / Serper
Version Control	Git
Environment Management	dotenv

- **Python –** Main programming language used to build the backend logic of the project. It connects all components like agents, APIs, and memory systems.
- **Streamlit –** Used to create a simple web-based user interface. It allows users to enter queries and view responses in a chat format.
- **CrewAI –** Framework used to manage multiple AI agents in the system. It controls task flow and coordinates how agents interact with tools.
- **Groq API (Llama 3.1 8B) –** Provides the language model that understands queries and generates responses. It performs reasoning and produces accurate answers quickly.
- **FAISS –** A vector database used to store past data as embeddings. It helps in retrieving similar information quickly using similarity search.
- **SentenceTransformers –** Converts text into numerical vector embeddings. These embeddings are used by FAISS for efficient searching and comparison.
- **Tavily / Serper –** Web search APIs used to fetch real-time information. They help the system provide up-to-date and relevant answers.
- **Git –** Version control system used to manage and track code changes. It helps in collaboration and maintaining different versions of the project.
- **dotenv –** Used to store environment variables like API keys securely. It keeps sensitive information separate from the main code.

3. Results & Output

3.1 Screenshots / outputs



➢ Key UI Components

- Header: Shows *Legal AI Assistant* title and tagline
- Sidebar: Clear chat option + query hints
- Input Box: Users enter legal questions
- Response Area: Displays AI-generated legal answers
- Suggestions: Shows related legal queries

➢ System Output Behavior

- The assistant gives **accurate and informative legal explanations**.
- It simplifies complex legal language into **easy-to-understand text**.
- It can also provide **contextual suggestions** for further queries.
- The system ensures **fast responses** using optimized LLM and agent workflow.

3.2 Key Observations

- User-friendly and clean UI
- Real-time legal query handling
- Context-aware intelligent responses
- Interactive chat-based experience
- Fast response time due to optimized AI model
- Accurate and research-based legal information
- Reduces manual legal research effort
- Memory-based learning for repeated queries
- Scalable and modular architecture
- Easy to use for non-technical users

4. Conclusion

- Demonstrates the use of **multi-agent AI systems** for solving real-world legal queries
- Integrates **CrewAI, Groq LLM, web search, and FAISS memory** effectively
- Provides **accurate, research-based, and easy-to-understand legal answers**
- Shows practical implementation of **agent-based AI workflows**
- Utilizes **vector databases for efficient memory and retrieval**
- Enhances understanding of **LLM integration and intelligent automation**
- Represents a strong and scalable **Agentic AI project solution**

5. Future Scope & Enhancements

- Add PDF legal document analysis
- Add voice-based query support
- Provide source citations for legal answers
- Implement user login and query history
- Replace FAISS with persistent vector databases
- Improve legal reasoning with domain-specific legal LLMs
- Add real-time legal case database integration